

RNN

```

import torch
import torch.nn as nn
import torch.optim as optim

vocab_size = 1000
embedding_dim = 16
hidden_size = 32
num_classes = 2
batch_size = 4
seq_length = 10

class ExtendedRNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embedding_dim)
        self.rnn = nn.RNN(embedding_dim, hidden_size, num_layers=2, bidirectional=True, batch_first=True)
        self.fc = nn.Linear(hidden_size * 2, num_classes)

    def forward(self, x):
        embedded = self.embedding(x)
        out, _ = self.rnn(embedded)
        return self.fc(out[:, -1, :])

model = ExtendedRNN()
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.01)

x = torch.randint(0, vocab_size, (batch_size, seq_length))
y = torch.randint(0, num_classes, (batch_size,))

optimizer.zero_grad()
predictions = model(x)
loss = criterion(predictions, y)
loss.backward()
optimizer.step()

print("--- Extended RNN Results ---")
print(f"Input shape: {x.shape}")
print(f"Predictions shape: {predictions.shape}")
print(f"Loss: {loss.item():.4f}")

```

```

--- Extended RNN Results ---
Input shape: torch.Size([4, 10])
Predictions shape: torch.Size([4, 2])
Loss: 0.6797

```

LSTM

```

import torch
import torch.nn as nn

vocab_size = 5000
embed_size = 64
hidden_size = 128
batch_size = 2
seq_len = 15

class ExtendedLSTM(nn.Module):
    def __init__(self):
        super().__init__()
        self.embed = nn.Embedding(vocab_size, embed_size)
        self.lstm = nn.LSTM(embed_size, hidden_size, num_layers=3, dropout=0.2, batch_first=True)
        self.fc = nn.Linear(hidden_size, vocab_size)

    def forward(self, x):
        embedded = self.embed(x)
        out, (h_n, c_n) = self.lstm(embedded)
        logits = self.fc(out)
        return logits, h_n, c_n

model = ExtendedLSTM()
x = torch.randint(0, vocab_size, (batch_size, seq_len))

logits, h_n, c_n = model(x)

print("--- Extended LSTM Results ---")

```

```
print(f"Input shape: {x.shape}")
print(f"Logits shape (Batch, Seq_Len, Vocab): {logits.shape}")
print(f"Hidden State shape: {h_n.shape}")
print(f"Cell State shape: {c_n.shape}")

--- Extended LSTM Results ---
Input shape: torch.Size([2, 15])
Logits shape (Batch, Seq_Len, Vocab): torch.Size([2, 15, 5000])
Hidden State shape: torch.Size([3, 2, 128])
Cell State shape: torch.Size([3, 2, 128])
```

BERT

```
import torch
from transformers import BertTokenizer, BertModel

tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
model = BertModel.from_pretrained('bert-base-uncased')

text = ["Natural language processing is fascinating.", "I need to debug this code."]
inputs = tokenizer(text, padding=True, truncation=True, return_tensors="pt")

with torch.no_grad():
    outputs = model(**inputs)

last_hidden_states = outputs.last_hidden_state
cls_embeddings = outputs.pooler_output

print("--- Extended BERT Results ---")
print(f"Input IDs shape: {inputs['input_ids'].shape}")
print(f"Attention Mask shape: {inputs['attention_mask'].shape}")
print(f"Last Hidden State (Word Embeddings) shape: {last_hidden_states.shape}")
print(f"CLS Token Embedding (Sentence Meaning) shape: {cls_embeddings.shape}")
```

```
tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 1.18kB/s]
vocab.txt: 232k/? [00:00<00:00, 3.87MB/s]
tokenizer.json: 466k/? [00:00<00:00, 9.65MB/s]
config.json: 100% 570/570 [00:00<00:00, 27.5kB/s]
model.safetensors: 100% 440M/440M [00:04<00:00, 172MB/s]
Loading weights: 100% 199/199 [00:00<00:00, 458.18it/s, Materializing param=pooler.dense.weight]
```

BertModel LOAD REPORT from: bert-base-uncased

Key	Status		
cls.predictions.transform.dense.bias	UNEXPECTED		
cls.seq_relationship.weight	UNEXPECTED		
cls.predictions.transform.LayerNorm.weight	UNEXPECTED		
cls.predictions.bias	UNEXPECTED		
cls.predictions.transform.dense.weight	UNEXPECTED		
cls.predictions.transform.LayerNorm.bias	UNEXPECTED		
cls.seq_relationship.bias	UNEXPECTED		

Notes:

- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch

--- Extended BERT Results ---

```
Input IDs shape: torch.Size([2, 11])
Attention Mask shape: torch.Size([2, 11])
Last Hidden State (Word Embeddings) shape: torch.Size([2, 11, 768])
CLS Token Embedding (Sentence Meaning) shape: torch.Size([2, 768])
```

RoBERTa

```
import torch
import torch.nn.functional as F
from transformers import RobertaTokenizer, RobertaForSequenceClassification

tokenizer = RobertaTokenizer.from_pretrained('cardiffnlp/twitter-roberta-base-sentiment')
model = RobertaForSequenceClassification.from_pretrained('cardiffnlp/twitter-roberta-base-sentiment')

texts = ["The new design is absolutely stunning!", "The server crashed again during the update."]
inputs = tokenizer(texts, padding=True, truncation=True, return_tensors="pt")

with torch.no_grad():
    outputs = model(**inputs)

logits = outputs.logits
probabilities = F.softmax(logits, dim=1)
```

```
label_map = {0: "NEGATIVE", 1: "NEUTRAL", 2: "POSITIVE"}

print("--- Extended RoBERTa Results ---")
for i, text in enumerate(texts):
    pred_class = torch.argmax(probabilities[i]).item()
    confidence = probabilities[i][pred_class].item()

    print(f"Text: '{text}'")
    print(f"Prediction: {label_map[pred_class]}")
    print(f"Confidence: {confidence:.4f}")
    print(f"Probabilities (Neg, Neu, Pos): {[round(p, 4) for p in probabilities[i].tolist()]}\\n")
```

Loading weights: 100%

201/201 [00:00<00:00, 510.83it/s, Materializing param=roberta.encoder.layer.11.output.dense.weight]

RobertaForSequenceClassification LOAD REPORT from: cardiffnlp/twitter-roberta-base-sentiment

Key	Status	
roberta.embeddings.position_ids	UNEXPECTED	

Notes:

- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch

--- Extended RoBERTa Results ---

Text: 'The new design is absolutely stunning!'

Prediction: POSITIVE

Confidence: 0.9887

Probabilities (Neg, Neu, Pos): [0.0017, 0.0096, 0.9887]

Text: 'The server crashed again during the update.'

Prediction: NEGATIVE

Confidence: 0.8911

Probabilities (Neg, Neu, Pos): [0.8911, 0.1034, 0.0055]

Start coding or [generate](#) with AI.